Beyond Capacity: Scalable MoE LLM Inference via High-Bandwidth Flash with Direct GPU and HBM Paths

Seeyeon Kim*, Juhyeong Jin* and Joo-Young Kim
KAIST
Daejeon, Republic of Korea
{seeyakim, pearlbro, jooyoung1203}@kaist.ac.kr

arXiv:2608.14333v1 [cs.AR] 14 Aug 2026

***Abstract*—Modern mixture-of-experts (MoE) language models increasingly strain the capacity and cost efficiency of high-bandwidth memory (HBM), as rapidly growing expert weights must be provisioned close to GPUs. High-bandwidth flash (HBF) offers substantially greater capacity, but conventional designs typically deliver HBF-resident expert weights to the GPU through HBM, leaving an additional direct GPU–HBF connection underutilized. We explore an HBF organization that simultaneously exploits two independent expert-delivery routes: a direct path that transfers expert weights from HBF to the GPU and a relay path that transfers them from HBF through the HBM base die to the GPU. Whole experts are assigned to one of the two routes, and transfers over both routes proceed concurrently, increasing aggregate expert-delivery bandwidth without replicating expert weights or introducing a shared relay bottleneck. Early expert determination identifies upcoming experts ahead of their conventional execution point, allowing HBF read latency to overlap with preceding computation, while separate management of immutable expert weights and mutable KV-cache data reduces interference between the two traffic classes. We evaluate the architecture using an event-driven continuous-batching LLM serving simulator with empirically measured GPU compute latencies. Across representative MoE workloads, concurrently utilizing the direct GPU–HBF and HBF–HBM–GPU routes consistently improves expert-delivery efficiency over designs restricted to either route alone. For a representative workload, the proposed architecture can achieve 1.94× higher throughput and 1.90× end-to-end speedup over a design that delivers all HBF-resident expert weights to the GPU through the HBM base die.**

I. Introduction

Modern LLM inference puts rapidly growing pressure on memory capacity and bandwidth as model sizes and context lengths increase. MoE models increase total parameter capacity by incorporating many experts, while limiting per-token computation to a small subset of activated experts [8], [16], [44]. Meanwhile, long-context requests generate larger key–value (KV) caches that must be retained throughout decoding [5], [23]. Continuous batching interleaves the prefill and decode phases of multiple requests, causing their KV caches to coexist in memory [1], [52]. These requests also invoke MoE layers with token-dependent expert selections, requiring the full expert set to remain stored and the selected weights to be delivered at each step. Together, these trends increase two major components of the memory footprint:

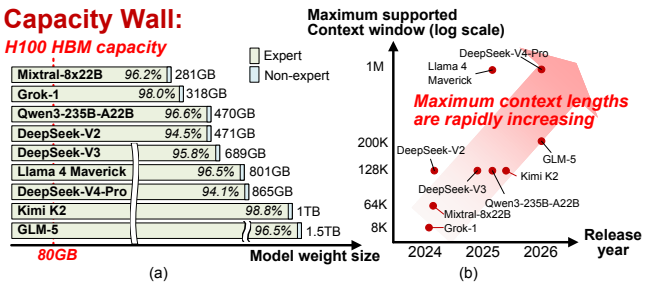

Fig. 1. (a) Model weight sizes and expert-weight fractions and (b) supported context-window lengths, derived from the published model configurations [5]–[7], [19], [23], [26], [35], [49], [53].

model weights grow with the total parameter count, whereas KV cache size grows with context length and batch size. Figure 1(a) illustrates this capacity pressure, showing that the surveyed MoE checkpoints have weight footprints of 281 GB–1.5 TB, with expert weights accounting for 94.1–98.8% of the total weight. Each model's weights alone exceed the NVIDIA H100's 80 GB HBM capacity [29], even before accounting for KV caches or activations. Figure 1(b) shows supported context windows reaching hundreds of thousands to millions of tokens, making KV cache and expert weight an increasingly significant memory component under long contexts and high concurrency.

High Bandwidth Flash (HBF) [37] offers a promising approach to alleviating this **capacity wall**. HBF is a die-stacked NAND memory that combines high capacity with aggregate read bandwidth from concurrent accesses across multiple dies and planes. Prior works [9], [33] have explored integrating HBF into the GPU memory hierarchy to leverage its capacity and bandwidth potential. However, such approaches retain an HBM-centric architecture, with HBF primarily serving as a backing or capacity-extension tier. Its cascaded GPU–HBM–HBF path requires HBF traffic to traverse HBM-side routing resources before reaching the GPU. Consequently, HBF traffic shares the HBM-side delivery path, preventing HBF bandwidth from being exposed to the GPU as an independent resource.

Direct GPU access, however, leaves two NAND-induced latency challenges. First, each read must sense a page from

*Both authors contributed equally to this research.

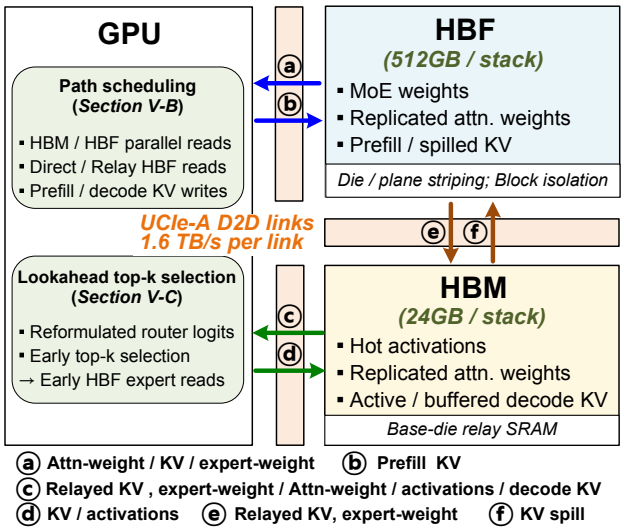

Fig. 2. Overview of DASH.

the NAND array into a page buffer before data transfer begins, incurring a startup latency of t_R [20], [51]. For dense layers, weight accesses follow a deterministic execution order and can therefore be prefetched [9]. In MoE models, however, expert selection is input-dependent, and the selected experts are not known until routing completes, making conventional prefetching ineffective. Prior work [33] avoids this problem by retaining expert weights in HBM, but the expert weights of recent large-scale MoE models [6], [7], [23], [26], [35], [49] can exceed the available HBM capacity.

Second, HBF program operations are much slower than reads and can delay latency-critical reads [2], [20]. Prefill generates large KV writes that can be programmed directly, whereas decode produces small updates that must be buffered and coalesced into pages.

To address these issues, we propose **DASH** (**D**irect **A**ttachment of HBF to the GPU as the main memory tier, with a **S**eparate path to **H**BM), as illustrated in Figure 2. DASH attaches both HBM and HBF to the GPU through independent UCIe links [41], allowing each memory to serve GPU requests without relying on the other. DASH also connects the two memory base dies through a dedicated HBM–HBF path. This path supports HBM-to-HBF writeback and relays HBF-resident reads through the HBM base die without accessing memory cells. Thus, the direct GPU–HBF path and the HBF–HBM-base-die–GPU relay path can operate concurrently, increasing delivery bandwidth for HBF-resident data such as MoE expert weights.

To exploit these paths for large MoE LLM inference, DASH places data across HBM and HBF according to its footprint, access pattern, and write behavior. Frequently updated data and intermediate activations are retained in HBM, whereas large read-mostly data is placed in HBF. Smaller non-expert weights reused by every token, such as attention QKV and output projection weights, are replicated across HBM and HBF. Within HBF, model weights and KV cache are distributed across multiple dies and planes to expose the available parallelism. We also separate model weights from KV cache at erase block granularity, preventing KV garbage collection from relocating model weight pages. DASH determines the top-k experts early and initiates their HBF reads before conventional routing completes, hiding t_R. Large prefill KV is written directly to HBF, whereas small decode updates are buffered and coalesced in HBM to hide program latency, t_{PROG}.

Together, these mechanisms match HBF access to the distinct bandwidth, timing and write-granularity demands of sparse expert delivery under dynamic serving. Therefore, DASH elevates HBF beyond a capacity-extension tier, making it a first-class component of the GPU memory system for large-scale MoE inference. With a batch size of 4, a 1K-token prefill, and 128 decode tokens, DASH delivers five-model geometric-mean throughput speedup of $1.90\times$ and reduces end-to-end latency by 44.8% over a cascaded-only baseline (RelayOnly), which delivers HBF data to the GPU through the HBM base-die relay path. Under continuous batching at 75% of this baseline's measured saturation rate, DASH reduces Qwen3 P90 end-to-end latency by 53.5%, compared with 53.3% against a direct GPU–HBF baseline (DirectOnly). In summary, this paper makes the following contributions:

- We elevate HBF from a passive capacity extension to a main memory tier by combining direct GPU attachment with a dedicated HBM–HBF path, enabling concurrent dual-path delivery of HBF-resident data.
- We hide HBF access latency by initiating selected expert reads as soon as lookahead selection completes and scheduling KV writes according to their production and reuse timing.
- We integrate DASH into a continuous-batching serving simulator that co-schedules decode tokens and chunked prefills at iteration boundaries, improving throughput and tail latency under dynamic arrivals.
- We establish DASH's system-level scalability, combining robust capacity-per-cost gains with a modular HBM–HBF topology.

II. Background

A. Modern MoE LLM Inference

1) Transformer Layer: Figure 3 shows an MoE Transformer layer [8], [44], [47], which consists of an attention block followed by an MoE block, with RMS normalization [54] and residual connections [10]. When processing individual requests, MoE inference combines two distinct memory behaviors. The attention block [47] performs QKV and output projections while reading and updating the KV cache, whose capacity grows with context length despite the reduced number of KV heads in GQA [3] and MQA [43]. The MoE block [8], [44] activates only the top-k experts for each token, reducing computation but not the capacity required to keep all expert weights addressable. Because routing follows

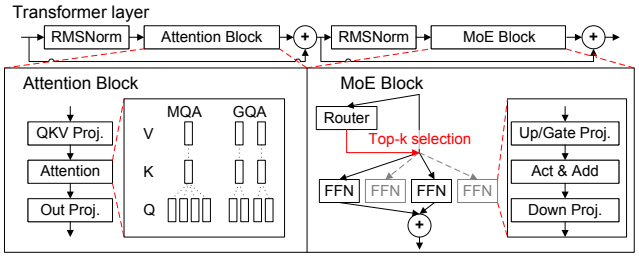

Fig. 3. Structure of MoE Transformer layer

TABLE I
COMPARISON OF MEMORY-CAPACITY EXPANSION APPROACHES

Approach	Capacity / bandwidth	Decode-path issue	Main limitation	Cost
Multi-GPU	Yes / High	GPU traffic overhead	Extra GPUs	High
CPU offload	Yes / Low	PCIe transfer bottleneck	Host-memory stalls	Low to medium
SSD offload	Yes / Low	NVMe access latency	On-demand read stalls	Low
HBF	Yes / Moderate	Read/program latency	Write cost / endurance	Not reported

attention, the selected experts are input-dependent and become known late. Thus, each request combines a growing, mutable KV working set with a large, read-mostly expert-weight set that is accessed sparsely and late.

2) Continuous Batching Serving: These per-request memory characteristics become more dynamic in online serving, where requests with different prompt and generation lengths arrive asynchronously. To maintain GPU utilization, modern serving systems commonly use continuous batching, admitting and removing requests at token boundaries rather than waiting for a fixed batch to complete [1], [52]. As requests enter and leave, the active batch, prefill/decode mixture, aggregate KV footprint, and union of selected experts continuously change [1], [44], [52]. The memory system must therefore manage large expert weights and growing mutable KV state under time-varying demand.

B. High-Bandwidth Flash

a) HBF Structure: High-Bandwidth Flash (HBF) is a stacked memory designed to combine the high storage density of NAND flash with high aggregate read bandwidth. HBF stack vertically integrates multiple flash core dies above a logic base die, with the dies interconnected by through-silicon vias (TSVs). Because each die contains multiple NAND planes, page reads can proceed in parallel across dies and planes [37], [51]. This structure can provide hundreds of gigabytes of capacity (e.g., 512 GB) and a reported stack-level read bandwidth of 1.6 TB/s [37], making HBF attractive for storing large model weights and KV cache.

b) HBF Characteristics: HBF retains NAND semantics: reads and programs operate on pages, erasure occurs at block granularity, and updates require out-of-place writes followed by garbage collection [2], [20]. Each HBF read incurs NAND sensing latency t_R before data transfer begins (e.g., 1–30 μs), favoring large parallel transfers over fine-grained random accesses [20].

Predictable accesses can hide t_R through prefetching and double buffering. In contrast, it may become exposed when the target address is determined late or when requests are too small to exploit parallelism across dies and planes. HBF programming has substantially higher latency and lower bandwidth than reading, with t_{PROG} typically ranging from tens to hundreds of microseconds (e.g., 100 μs), so fine-grained writes can stall inference [20].

Finally, finite program/erase endurance makes sustained writes a lifetime concern [11]. Read-only model weights cause little write wear, whereas frequently updated KV cache requires efficient write scheduling and endurance management.

III. Motivation

A. Limitations of Existing Capacity Scaling

As shown in Table I, CPU-based and SSD-based offloading schemes place model weights or KV cache blocks in host memory and storage respectively. Tightly coupled CPU–GPU systems such as Grace Hopper substantially increase GPU–host-memory bandwidth through NVLink-C2C [30], but the bandwidth remains well below the GPU's local HBM bandwidth.

Multi-GPU model parallelism expands memory capacity by aggregating GPU memory, but necessarily scales compute capacity with it. Although Qwen3-235B-A22B [35] activates only 22B parameters per token, storing its full 235B-parameter BF16 weight set requires approximately 470 GB, nearly exhausting the nominal 480 GB aggregate HBM2e capacity of six 80-GB NVIDIA H100 PCIe GPUs [28] before reserving memory for the KV cache, activations, communication buffers, and runtime workspace. Because autoregressive decoding is often memory-bandwidth bound, GPUs added primarily to satisfy memory-capacity requirements can leave much of their compute underutilized [1], [34]. Tensor parallelism incurs communication and synchronization overhead from inter-GPU collectives, whereas pipeline parallelism adds inter-stage transfers and pipeline bubbles [27], [34]. In MoE models, input-dependent routing can further create load imbalance by concentrating tokens on experts hosted by particular GPUs [14]. Thus, scaling out H100 GPUs merely to fit model weights increases system overhead even when the added compute is not fully utilized.

B. Latency Challenges in HBF

Direct GPU access alone is insufficient to use HBM and HBF efficiently. Data placement must balance traffic across both memories, expose die- and plane-level HBF parallelism, and separate long-lived model weights from frequently updated KV cache. Each HBF read incurs the NAND sensing

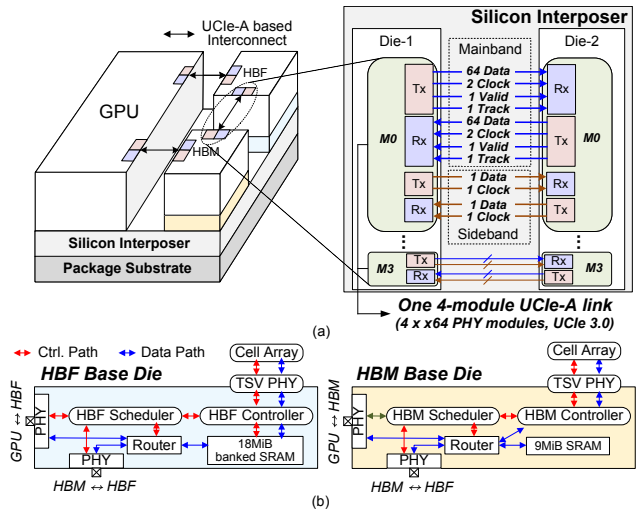

Fig. 4. (a) DASH architecture and a four-module UCIe 3.0 UCIe-A link provisioned for 1.6 TB/s of usable D2D bandwidth; (b) microarchitectures of the HBF and HBM base dies.

latency, t_R, before data transfer begins. Predictable dense-layer accesses can hide t_R through prefetching, whereas MoE expert accesses depend on routing results. If an expert read is issued only after routing completes, t_R is exposed on the inference critical path. Expert selection must therefore begin early enough to overlap t_R with preceding computation. HBF writes incur the substantially longer page-program latency, t_{PROG}. Prefill produces large KV cache bursts that can be combined into page-sized writes, whereas decode generates small per-token updates that poorly utilize the program granularity. Issuing these writes immediately would repeatedly expose t_{PROG} and interfere with HBF reads.

Our design addresses these challenges using independent paths, HBF-aware data placement, early expert selection to hide t_R, and phase-aware KV buffering and scheduling to mitigate the impact of t_{PROG}.

IV. DASH Architecture

We propose DASH, a heterogeneous near-GPU memory architecture that integrates HBM and HBF. Figure 4(a) shows the overall architecture of DASH, where HBM and HBF operate as independent near-GPU memories. DASH provides three data paths: GPU–HBM, GPU–HBF, and HBM–HBF. These paths are implemented using Universal Chiplet Interconnect Express (UCIe) links that connect the GPU I/O die to the HBM and HBF base dies and directly connect the two memory base dies [4], [46].

A. UCIe-Based Unified Memory Interconnect

DASH equips the GPU I/O die and both memory base dies with UCIe 3.0 physical layers (PHYs) and die-to-die (D2D) adapters, providing a common transport across the three data paths [41], [42], [46]. A common transaction format carries the operation, address, transfer length, and data over the UCIe mainband, while the receiving scheduler routes each request to the appropriate memory controller, which translates it into HBM- or NAND-specific commands [4], [40], [46].

Each path is implemented as one four-module UCIe-A link, whose common adapter and multi-module PHY logic coordinate four paired $\times 64$ modules at 64 GT/s as a single logical link [41], [42], [46]. The resulting 2.048 TB/s raw capacity per direction leaves approximately 22% headroom for link and implementation losses while supporting 1.6 TB/s of modeled usable D2D streaming bandwidth per physical link [41], [46]. Each module retains an independent sideband for link training, configuration, and repair, whereas the common adapter presents one transaction stream to the base-die router [46]. NAND sensing and shared-TSV SRAM fills are accounted for separately from this D2D rate and overlapped through double buffering.

Building on UCIe-based attachment of memory base dies [4], [40], DASH relays HBF reads through the HBM base-die router to the GPU-facing link without accessing the HBM controller or cell array. We refer to this end-to-end HBF–HBM-base-die–GPU route as the *Relay path*. UCIe 3.0 defines UCIe-A energy-efficiency targets of approximately 0.5 pJ/bit at 0.5 V and 0.6 pJ/bit at 0.7 V for the relevant high-speed operating points, and specifies a 0.389-mm-wide UCIe-A module [46]. Four such modules therefore require roughly 1.6 mm of die-edge width per endpoint [42], [46]. At 1.6 TB/s, these energy-efficiency targets correspond to approximately 6.4–7.7 W of link power per fully utilized direction [46]. These specification-based estimates characterize the die-edge footprint and link power, while physical design further accounts for routing, signal- and power-integrity, and power-delivery costs [4], [41], [42], [48].

B. HBM and HBF Base-Die Microarchitecture

Figure 4(b) illustrates the base-die microarchitecture of HBM and HBF of DASH architecture.

a) HBF Base Die: The HBF base die handles read and write requests received from the GPU or the HBM side through UCIe. It contains an HBF controller, a local scheduler, and 18 MiB of physical banked SRAM per stack. Of this capacity, 16 MiB is usable data storage organized into two interleaved 8 MiB transfer regions. The additional 2 MiB accounts for 12.5% ECC check-bit storage, assuming eight check bits per 64 data bits. Each transfer region matches one stack-local page wave in our evaluation. The two transfer regions alternate between shared-TSV fill and D2D drain. Each read still incurs t_R before its data become ready in SRAM. Independently accessible banks allow different ready chunks to drain concurrently over the GPU–HBF path and *Relay path*. The scheduler issues a request only when the target die or plane, sufficient SRAM space, and the required transfer path are available. The HBF controller then converts the request into NAND read or program commands. Incoming writes use an available transfer region and are backpressured when both regions are occupied.

b) HBM Base Die: The HBM-side relay stream is segmented into 4 MiB transfer chunks, each equal to half of the 8-MiB stack-local HBF page wave. Each HBM base die provisions 9 MiB of physical banked SRAM per stack. Of this capacity, 8 MiB is usable relay storage organized as two interleaved 4-MiB transfer regions. The additional 1 MiB accounts for the same 12.5% ECC check-bit storage. The two transfer regions alternate between receiving one chunk from HBF and forwarding the preceding chunk to the GPU. Because the HBF–HBM and GPU–HBM links have the same 1.6 TB/s rate, this organization sustains the *Relay path* after the initial fill. When both regions are occupied, the scheduler withholds credits for *Relay path* until one region becomes available. The *Relay path* data bypasses the HBM controller and cell array, so this transfer does not access DRAM. Small buffer metadata and request and credit queues are provisioned separately and are not included in the quoted SRAM capacities.

Overall, DASH combines independent GPU access to HBM and HBF with direct data movement between the two memories. This architecture allows HBF data to use the GPU–HBF path alone or additionally use the GPU–HBM path through the *Relay path*.

V. Placement and Execution

A. Data Placement

1) Placement Along Architecture: DASH places data according to size, mutability, and reuse: HBM holds frequently updated state, HBF holds large read-mostly data, and small frequently reused read-only weights may be replicated. Expert weights and write-once/read-many prefill KV reside in HBF, while smaller attention weights are replicated across HBM and HBF. Fine-grained decode KV is initially accumulated in HBM and migrated to HBF in complete page-aligned units before HBM fills.

2) Placement Within HBF: For MoE weights, mapping each expert to a single plane or a small subset of planes would limit the available read bandwidth. DASH therefore divides each expert's weights into chunks and distributes them across multiple HBF dies and planes. As shown in Figure 5(a), this placement allows the weights of a selected expert to be read in parallel and utilize high HBF read bandwidth even when only a few experts are activated.

Because HBF erases at block granularity, garbage collection may relocate live pages [2]. To isolate model weights from KV garbage collection, as illustrated in Figure 5(b), DASH stores model weights and KV cache in separate weight-owned and KV-owned blocks. During garbage collection, a KV-owned block is erased directly if it contains no live KV pages. This separation allows KV garbage collection to proceed without examining or relocating model weight pages, reducing data-relocation traffic.

B. Dataflow and Execution

The GPU-side scheduler routes each request to the memory holding its data through an available path with buffer space. Figure 6(a) summarizes four representative execution modes.

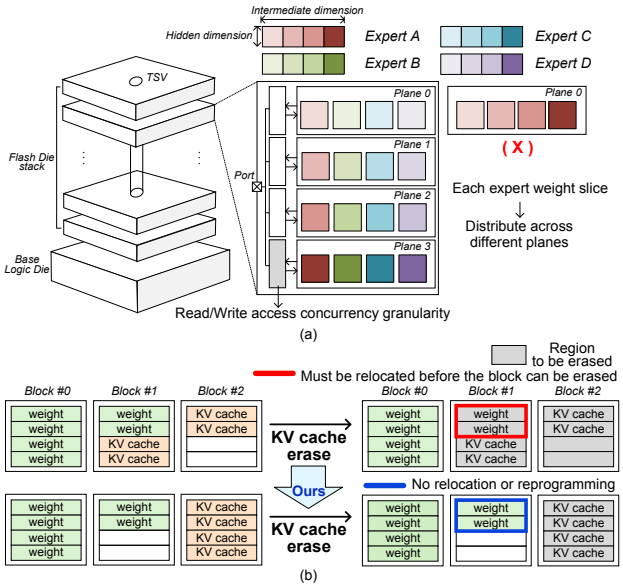

Fig. 5. (a) Placement of MoE expert weights across HBF planes for parallel access (b) KV cache placement with erase block granularity

a) Parallel HBM/HBF Read: In this mode, the GPU concurrently reads distinct data from HBM and HBF through their respective paths. First, different portions of the replicated attention weights can be read from HBM and HBF in parallel, increasing the aggregate bandwidth available to the GPU. Second, attention can simultaneously read the KV cache ranges placed in HBM and HBF. The GPU computes partial attention results for each range and combines them before producing the final attention output.

b) Dual-Path HBF Read: The dual-path HBF read uses independent SRAM banks to concurrently deliver different HBF-resident chunks over both GPU-facing paths. NAND reads still incur t_R, and each chunk becomes transferable only after it is filled into an SRAM bank through the shared TSV. Double buffering overlaps this preparation with transmission of previously ready chunks. Direct reads move from HBF-side SRAM to the GPU, while relayed reads traverse HBM-side SRAM and the GPU–HBM path without accessing the HBM cell array. Separate SRAM banks allow both paths to operate concurrently once their data are ready.

DASH uses this mode when additional delivery bandwidth is needed for selected expert weights or HBF-resident KV cache. The conventional design leaves one GPU-facing path idle while the other drains ready data, resulting in underutilized bandwidth and a longer completion time. As shown in Figure 6(b), DASH simultaneously drains different ready chunks over both GPU-facing paths to increase aggregate delivery bandwidth and reduce transfer latency.

c) Direct Prefill Write: During prefill, DASH writes newly generated KV cache directly from the GPU to HBF. Because prefill generates a large volume of KV cache at once, its writes achieve high utilization of the HBF program granu-

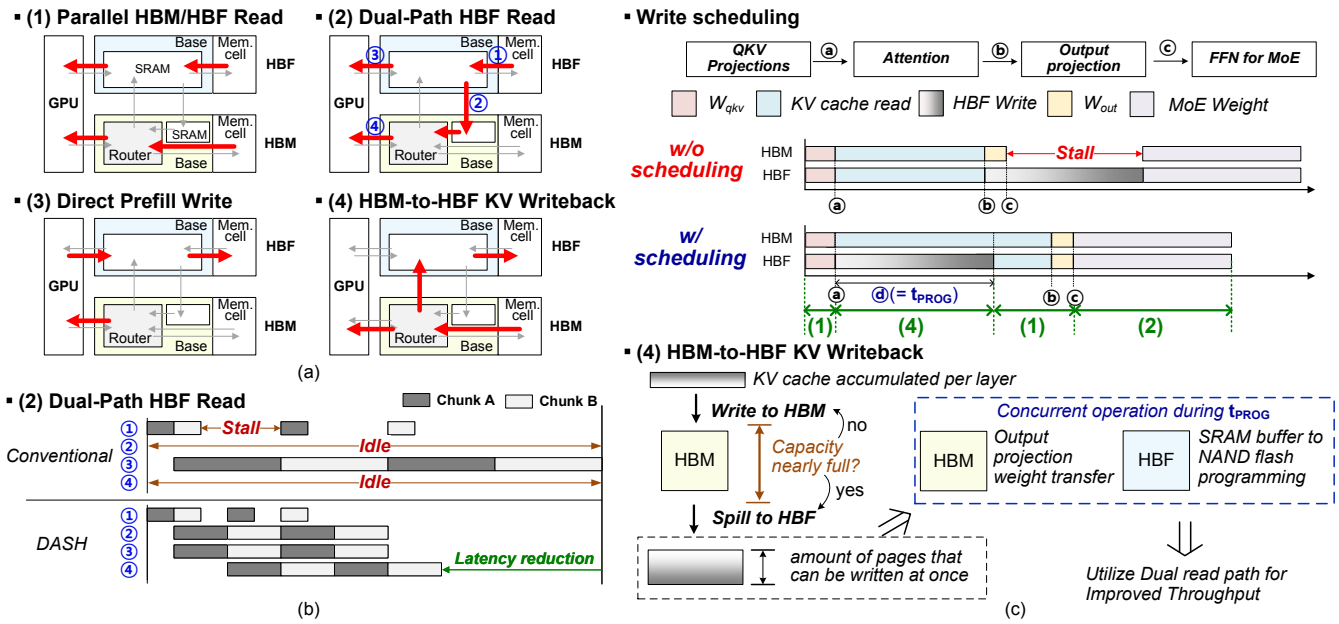

Fig. 6. (a) Four primary execution modes of DASH; (b) latency reduction with dual-path HBF reads; (c) effect of HBF write scheduling.

larity. The generated KV cache is sent through the GPU–HBF path to the SRAM of HBF and is then programmed into HBF by the HBF controller. Direct prefill write execution sends the persistent KV copy from the GPU to HBF without staging it in HBM. Because the GPU–HBM and GPU–HBF paths operate independently, the current layer's output-projection weights can be fetched from HBM while HBF programming overlaps the attention computation. The compute-intensive attention operation provides a long interval in which the HBF program latency can be hidden. This scheduling avoids serializing the prefill KV write with the subsequent attention computation.

d) HBM-to-HBF KV Writeback: Since HBF program latency is substantially longer than read latency, write timing is crucial. During decode, DASH appends small KV entries to HBM because individual entries are often too fine-grained for efficient HBF programming [2], [20]. A full HBF page wave is the aggregate data volume that can be programmed concurrently across all HBF stacks, dies, and planes. Once HBM approaches its capacity limit and each HBM stack has accumulated its partition of a full page wave, DASH writes the wave back in parallel over the HBM–HBF paths, as shown in Figure 6(c). The transfer begins after the current layer finishes its QKV projection, allowing the HBF program latency to overlap with the following attention computation. The attention computation uses the KV cache in HBM, while older KV entries are transferred to HBF through the HBM–HBF path. The HBM controller transfers the selected KV data to the HBF-side SRAM over the HBM–HBF path, and the HBF controller then writes it into HBF. After writeback, DASH records the KV data's new HBF location and can free the corresponding HBM space. Batching small decode updates into larger HBF writes avoids a separate programming delay on every decode iteration.

C. Lookahead Expert Execution

Although HBF provides high read bandwidth, each read incurs the NAND sensing latency t_R [20], [37], [51]. For predictable access sequences, DASH hides this latency using double buffering in the HBF base die. While one HBF-side SRAM buffer supplies the current data to the GPU, the other prepares data for the next access. For non-expert weights and scheduled KV cache accesses, upcoming addresses follow the model execution order and can be identified in advance.

However, this approach does not directly apply to MoE expert accesses [13], [55]. Because MoE routing is input-dependent, the selected experts are not known until routing completes [6]–[8], [17], [44]. In conventional execution, the attention block is followed by the output projection, residual addition, and RMSNorm, after which routing begins [6], [7], [17], [47], [54]. When expert weights reside in HBF, their reads cannot be issued until this point, exposing the initial HBF read latency [9], [13], [33], [51]. Prior work [33] avoids this latency by keeping MoE expert weights in HBM. However, the expert weights of large-scale MoE models can exceed the available HBM capacity, making this approach impractical [7], [23], [26], [32], [35]. Therefore, when expert weights are placed in HBF, the selected experts must be determined early enough to initiate their reads [22], [55].

DASH decomposes conventional routing into an input-only term computed before attention and an output-dependent term computed as soon as the attention output becomes available. Combining these terms determines the top-k experts before conventional routing, as shown in Figure 7(a).

Let X denote the input to the RMSNorm preceding the attention block, O the attention output before the output

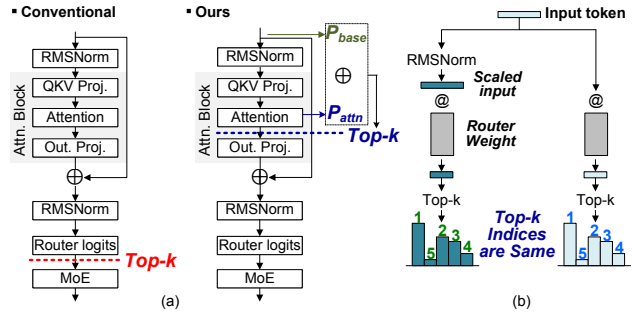

Fig. 7. (a) Comparison of MoE top-k selection timing (b) Equivalent top-k selection without waiting for RMS scaling

projection, W_{out} the output projection weight, γ the scale parameter of the RMSNorm preceding the MoE block, and W_{r} the routing weight. The conventional routing logits are

$$P = \text{RMSNorm}\left(X + OW_{\text{out}}\right) W_{\text{r}}. \tag{1}$$

Using the definition of RMSNorm [54], these logits can be rewritten as

$$P = \alpha \left(P_{\text{base}} + P_{\text{attn}}\right), \tag{2}$$

where

$$P_{\text{base}} = X \,\text{diag}(\gamma) W_{\text{r}}, \tag{3}$$

$$P_{\text{attn}} = O\widehat{W}_{\text{r}}, \qquad \widehat{W}_{\text{r}} = W_{\text{out}} \,\text{diag}(\gamma) W_{\text{r}}, \tag{4}$$

and

$$\alpha = \frac{1}{\text{RMS}\left(X + OW_{\text{out}}\right)}. \tag{5}$$

The base-routing term, P_{base}, depends only on X and can therefore be computed before the attention computation begins. The attention-routing term, P_{attn}, can be computed as soon as O becomes available, without waiting for completion of output projection. Because $\widehat{W}_{\text{r}} = W_{\text{out}} \,\text{diag}(\gamma) W_{\text{r}}$ is constant during inference, it can be precomputed and stored as an additional weight. Thus, DASH obtains $P_{\text{base}} + P_{\text{attn}}$ before the conventional routing point.

The scale factor α is a positive scalar shared by all expert logits of the same token. It changes their magnitudes but does not affect the rank. Therefore, selecting the top-k experts from $P_{\text{base}} + P_{\text{attn}}$ produces the same routing decision as conventional routing, as shown in Figure 7(b).

DASH therefore determines the top-k experts without waiting for the RMS value. The early routing terms are used only for expert selection. Once the RMSNorm output becomes available, the subsequent FFN operation can be done with the expert weight selected beforehand. This allows the HBF controller to begin reading the selected expert weights as soon as the attention-routing term completes [22], [55]. Conventional execution instead waits for the residual addition, RMSNorm, and routing computation, leaving HBF idle between the output projection and expert-weight streams [6], [7], [17]. DASH minimizes this read stall and extends HBF latency hiding to input-dependent MoE expert accesses.

TABLE II
EVALUATED MOE MODELS AND CONFIGURATIONS

Model	Attn.	Weights (GB)	Experts / Top-k	Weight Precision
Qwen3-235B-A22B	GQA	470.19	128+0 / 8	BF16
Mixtral-8×22B	GQA	281.24	8+0 / 2	BF16
Grok-1	GQA	318.24	8+0 / 2	INT8-mixed
Llama 4 Maverick	GQA	801.42	128+1 / 1	BF16
DeepSeek-V3	MLA	688.59	256+1 / 8	FP8-mixed
DeepSeek-V2	MLA	471.49	160+2 / 6	BF16

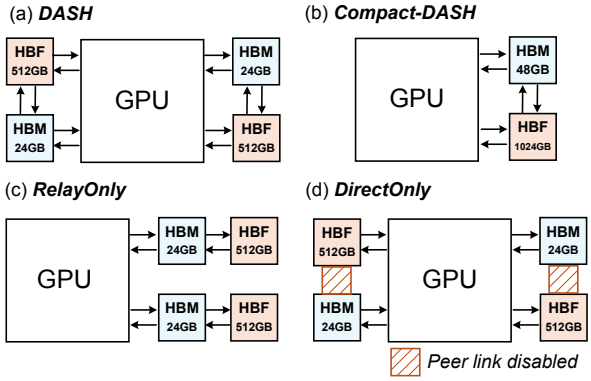

Fig. 8. Four configurations used in the evaluation.

DASH computes expert selection in FP32 to ensure accurate selection. It uses early selection only when routing is scale-invariant and free of expert-specific additive bias. For all other routers, including the router used in DeepSeek-V3, DASH performs conventional late selection while preserving its placement and dual-path delivery [7]. All other model computations remain at their native precision.

VI. EVALUATION

A. Methodology

a) Models: Table II summarizes six MoE model configurations [6], [7], [23], [26], [35], [49]. Five are evaluated in the main sweeps over batch size and sequence length, while DeepSeek-V2 is used separately for the Lookahead Expert Execution study described in Section VI-D. The Experts column lists the numbers of routed and shared experts.

b) Baselines and HBF Configuration: Figure 8 illustrates four architectural configurations used to evaluate the effectiveness of DASH. Across all configurations, we use the consistent data-placement policy: expert weights reside in HBF, QKV and output projection weights are replicated across HBM and HBF. The DASH configuration consists of two 512 GB HBF stacks and two 24 GB HBM stacks [25], [37], with each physical D2D link provisioned for 1.6 TB/s of usable streaming bandwidth. NAND/TSV preparation and SRAM readiness are modeled separately from this link rate. RelayOnly and DirectOnly preserve the same number of memory stacks while removing the direct GPU–HBF path and the HBF–HBM path, respectively. Compact-DASH retains

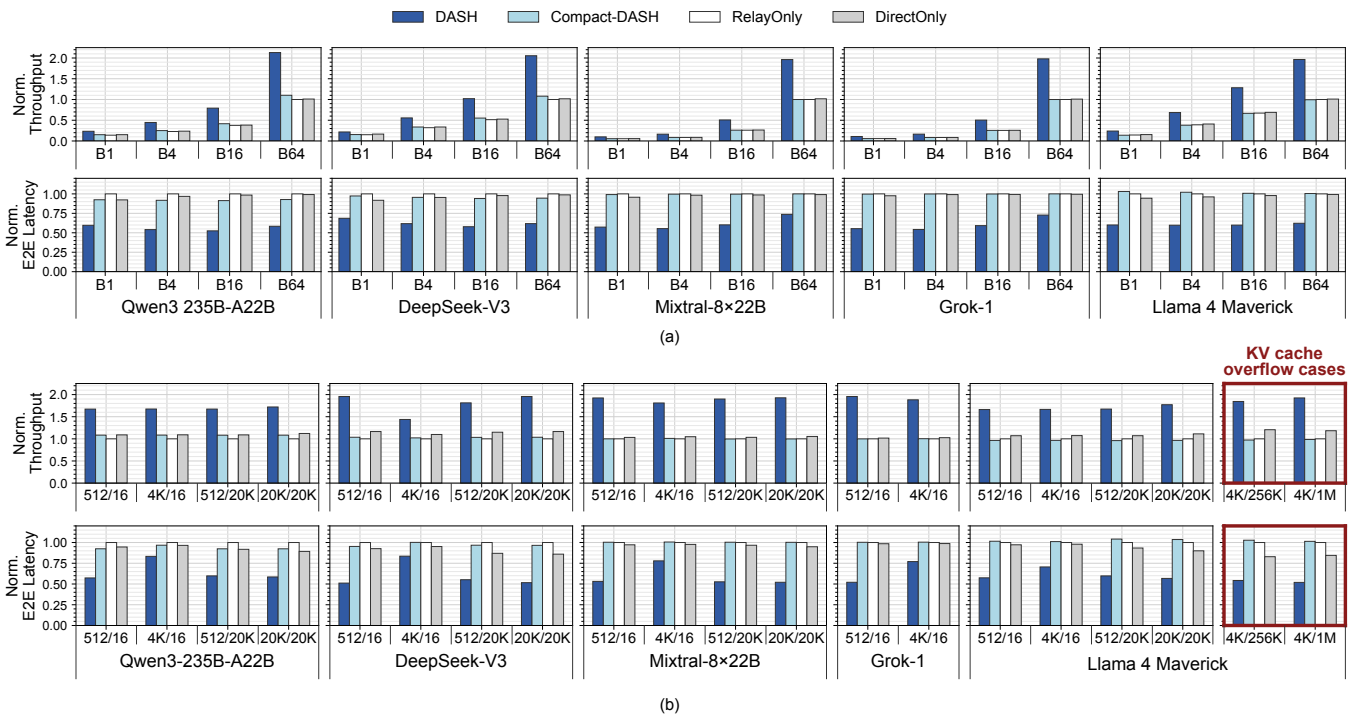

Fig. 9. Normalized throughput and E2E latency under (a) batch-size scaling and (b) B1 input/output sequence-length variation. In (b), each x-axis label reports $L_{\text{in}}/L_{\text{out}}$, the input/output sequence lengths. Red boxes mark the Llama 4 Maverick KV cache overflow cases.

both paths but consolidates the memory resources into a single HBM/HBF stack pair, enabling comparison under a matched GPU-side connectivity budget with area reduction.

We model 4-KiB HBF pages following the physical-page organization described in the HBF patent [51]. Because public HBF disclosures do not specify the exact subarray-level parallelism, we assume four independently accessible subarrays per plane. With 16 dies and 32 planes per die, this yields an 8 MiB page wave per stack and 16 MiB across two stacks. We use 3-μs read and 100-μs program latencies as nominal points and sweep these parameters in Section VI-F.

c) Simulation: To evaluate DASH, we extend LLMSimulator [38] with HBF modeling and shape-aware GPU performance profiling. The HBF model captures read and program latencies, die-level and plane-level parallelism, SRAM-buffer readiness, per-link D2D availability, and KV-cache writeback of adjacent paths. We profile GPU operator latencies on an NVIDIA H100 PCIe 80 GB GPU using the evaluated workloads [31]. The simulator directly uses measured latencies for profiled GPU operator shapes and interpolates unmeasured shapes only within validated per-operator ranges. Across 1,107 validation measurements of supported non-router operators, the GPU timing model achieves a median relative error of 0.51%, with 90% of errors below 3.52%.

We use B, L_{in}, and L_{out} to denote batch size and input/output sequence lengths, respectively. Unless otherwise stated, fixed-batch experiments use $B/L_{\text{in}}/L_{\text{out}} = 4/1\text{K}/128$.

B. Throughput and End-to-End Latency

Figure 9 compares the normalized throughput and E2E latency of DASH, Compact-DASH, RelayOnly, and DirectOnly. Figure 9(a) evaluates five MoE models at $B \in \{1, 4, 16, 64\}$ with $L_{\text{in}}/L_{\text{out}}$=1K/128. Figure 9(a) normalizes throughput to RelayOnly at B=64 for each model and E2E latency to RelayOnly at the corresponding batch size. Figure 9(b) fixes B=1 and varies the input/output sequence lengths, denoted by $L_{\text{in}}/L_{\text{out}}$: short $512/16$, prefill-dominant $4\text{K}/16$, decode-heavy $512/20\text{K}$, and long prefill+decode $20\text{K}/20\text{K}$. Figure 9(b) normalizes both metrics to RelayOnly for each model and workload. Figure 9(b) additionally evaluates Llama 4 Maverick KV cache overflow from HBM into HBF at $L_{\text{in}}/L_{\text{out}}$=4K/256K and 4K/1M. We include only workloads that fit within each model's supported context window; consequently, Grok-1 is evaluated only at $512/16$ and $4\text{K}/16$.

Figure 9(a) shows that DASH consistently outperforms RelayOnly and DirectOnly in both throughput and E2E latency at every batch size. Across all five models and batch sizes, DASH achieves geometric-mean throughput speedups of 1.90× over RelayOnly and 1.84× over DirectOnly, while reducing E2E-latency by 42.2% and 40.8%, respectively. DASH achieves these gains by transferring HBF-resident expert weights and KV cache over the Direct and Relay paths concurrently. Despite halving the HBM/HBF stack-pair count, Compact-DASH delivers performance comparable to RelayOnly and DirectOnly. At each MoE layer, selected-expert transfers can use either the GPU–HBF Direct path or the HBM-mediated

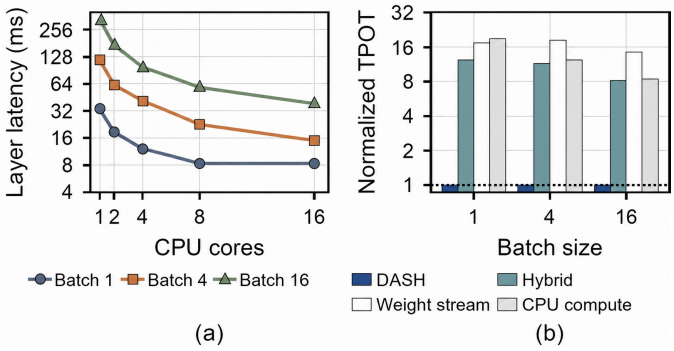

Fig. 10. CPU-aware offloading: (a) median integrated Expert-layer latency under 1–16 CPU-core caps; (b) median-based TPOT normalized to DASH. CPU compute and endpoint-guarded Hybrid use the same 32-core cap in (b).

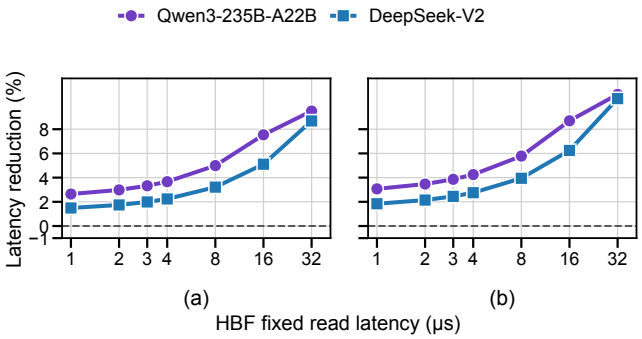

Fig. 11. Impact of early expert decision on (a) request E2E latency and (b) time per output token.

Relay path, enabling effective utilization of both transfer paths even within a single HBM/HBF pair. Consequently, Compact-DASH consistently outperforms the single-path layouts while providing a smaller, more area-efficient design point.

Figure 9(b) shows that DASH consistently delivers high throughput and low E2E latency across all evaluated workloads. Across the 20 combinations, DASH achieves geometric-mean throughput speedups of 1.79× and 1.63× over RelayOnly and DirectOnly, respectively, while reducing E2E latency by 40.1% and 35.6%, based on paired latency ratios. In particular, compared with the prefill-dominated $4\mathrm{K}/16$ cases, the long-decode $512/20\mathrm{K}$ and $20\mathrm{K}/20\mathrm{K}$ cases exhibit larger E2E latency reductions while sustaining similarly high throughput across models. Repeated decode steps create more opportunities for concurrent HBM–HBF access, while the proposed HBF write scheduling coordinates the growing KV cache traffic with latency-critical expert weight accesses.

The final two graphs evaluate Llama 4 Maverick with increasingly large KV caches. For shorter decode workloads, the generated KV cache remains in HBM and does not trigger HBM-to-HBF writeback. As the decode length increases, the KV cache grows beyond the available HBM capacity, causing older KV entries to be written back to HBF. DASH continues to achieve high throughput and low E2E latency even when HBM-to-HBF KV writeback occurs. Even for Llama 4 Maverick with a 197.413 GB KV cache generated by a long decode sequence, DASH achieves 1.92× higher throughput and reduces E2E latency by 48.0% over RelayOnly. This robustness stems from DASH's two-level KV management: HBM absorbs fine-grained per-token updates, while batching them into larger writebacks through write scheduling reduces the HBF programming overhead per KV entry and reclaims HBM capacity.

C. Comparison with CPU–GPU Offloading Strategies

When MoE weights exceed HBM, weight stream transfers each uncached expert's weights to the GPU [50], whereas CPU compute sends activations to CPU-resident experts [18]. We also evaluate a Hybrid oracle over all evaluated CPU/GPU workload splits, including the CPU-only and GPU-only configurations. For each split, we independently measure median TPOT; mixed splits execute CPU and GPU concurrently. We report the minimum, providing an empirical upper bound on Hybrid performance, or equivalently, a lower bound on median TPOT, within the evaluated split space.

We measure a shape-equivalent Qwen3 BF16 expert layer on an Intel Xeon Platinum 8452Y in the H100-local NUMA node using physical cores [15]. Figure 10 (a) sweeps 1–16-core caps, while CPU compute and Hybrid in Figure 10 (b) are independently tuned under the same 32-core cap. This oracle-favorable comparison combines measured expert-layer costs with the same non-expert GPU time across all schemes.

Figure 10 (a) shows that B1 saturates once its eight expert jobs are parallelized, whereas larger batches expose enough jobs to benefit from additional cores. By adapting the CPU/GPU split, Hybrid oracle reduces TPOT by 2.8–29.4% over the better single-path configuration across the evaluated batch sizes. Even with this favorable baseline, Hybrid remains 8.22–12.32× slower than DASH because DASH avoids both CPU execution and host-PCIe weight staging.

D. Impact of Lookahead Expert Execution

We isolate the effect of expert early decision within DASH by comparing it against the conventional execution flow. We measure the latency reduction at $B/L_{\mathrm{in}}/L_{\mathrm{out}} = 1/4\mathrm{K}/128$. Because concrete HBF specifications are not yet available, we further sweep the HBF read latency, t_R, from 1 to 32 μs to evaluate expert early decision across a range of plausible HBF configurations.

As shown in Figure 11, at 3 μs, expert early decision reduces E2E latency by 3.33% and 1.99%, and TPOT by 3.86% and 2.45% for Qwen3 and DeepSeek-V2, respectively. At 32 μs, the corresponding E2E reductions increase to 9.50% and 8.69%, while the TPOT reductions reach 10.88% and 10.53%, respectively. Lookahead Expert Execution exposes the target experts before conventional routing completes, allowing HBF read startup and weight delivery to overlap with the following output projection computation. Thus, increasing t_R initially

TABLE III
QWEN3 P90 LATENCY AT 1.6 TB/S PER PHYSICAL D2D LINK

Load	Metric	DASH P90	Reduction vs. RelayOnly	Reduction vs. DirectOnly
50%	E2E	20.982 s	61.2%	61.0%
	TPOT	148.0 ms	62.2%	62.0%
75%	E2E	28.847 s	53.5%	53.3%
	TPOT	208.5 ms	53.9%	53.8%
90%	E2E	31.859 s	50.5%	50.4%
	TPOT	232.0 ms	50.3%	50.2%

exposes more latency that can be hidden from the expert-computation critical path. Even when t_R exceeds the interval between early selection and expert-weight consumption, the delay within this interval can only be overlapped; the excess remains on the critical path, but the overlapped portion still provides a clear reduction in both E2E latency and TPOT.

E. Continuous-Batching Serving

We evaluate Qwen3-235B-A22B with 4,096-token prompts, 128 generated tokens per request, $B_{\max}$=32, an 8,192-token iteration budget, and 512-token prefill chunks. At each iteration boundary, the scheduler reserves one decode token per active request before admitting at most one prefill chunk per waiting request. Mixed co-schedules decode and prefill in one pass, whereas Serial uses separate decode and prefill passes, reusing each fetched expert across routed rows within a pass. For each mode, all configurations replay identical arrivals and use the same analytical routing with balanced per-expert rows, scheduling, placement, capacities, and H100 BF16 timings. DirectOnly carries HBF-resident experts over the two GPU–HBF links, whereas RelayOnly uses the two HBF–HBM–GPU relay routes; DASH uses both route sets concurrently. At 1.6 TB/s per link, the aggregate bandwidth for SRAM-ready expert data is up to 3.2 TB/s in either baseline and 6.4 TB/s in DASH. DASH assigns each active expert entirely to either Direct or Relay without splitting its weight tiles across the two. Because both route sets operate concurrently, expert-delivery time is $\max(T_{\text{Direct}}, T_{\text{Relay}})$. We replay five Poisson traces at 50%, 75%, and 90% of RelayOnly's backlogged throughput under Mixed, discarding 128 warm-up requests and measuring the next 1,024 requests per trace. Table III summarizes P90 TPOT and P90 E2E latency.

Under Mixed, DASH reduces both P90 metrics in every seed, load, and baseline comparison. Mixed raises peak throughput over Serial by 10.3% for DASH and 12.6% for the baselines, while DASH retains 34.1%–37.1% higher peak throughput than the single-route baselines across both modes.

F. Sensitivity Analysis

Figure 12 evaluates the sensitivity of DASH to key HBF parameters using Llama 4 Maverick at $B/L_{\text{in}}/L_{\text{out}} = 1/4\text{K}/1\text{M}$. The sweep uses asymmetric KV placement: the KV state produced by the 4K-token prefill is placed in HBF, whereas newly generated decode KV is appended to HBM.

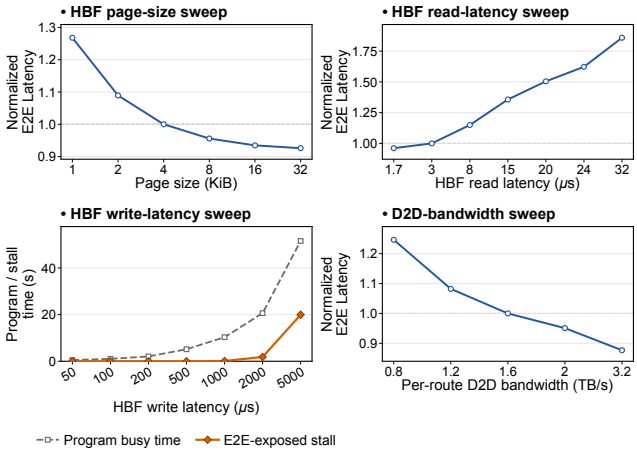

Fig. 12. HBF parameter sensitivity. All panels report normalized E2E latency except the program-latency panel, which reports cumulative program-busy and E2E-exposed stall time.

When the package-wide HBM-resident decode KV reaches the 24.46 GB decode-KV budget, the oldest page-aligned decode KV begins to spill from HBM to HBF. At the final context, 24.46 GB of decode KV is balanced across the two 24-GB HBM stacks (approximately 12.23 GB per stack), while the remaining 172.96 GB resides in HBF, for a total KV cache size of 197.41 GB. For the page-size sensitivity study only, we scale the transfer-region capacity in proportion to the bytes delivered per page wave.

For HBF page size and read latency, results are normalized to the 4 KiB and 3 μs configurations, respectively. As shown in Figure 12, a larger page size increases the amount of data transferred per HBF access, improving the effective aggregate read bandwidth and reducing E2E latency. Similarly, lower read latency allows more HBF reads to complete within a given time, further increasing the effective aggregate read bandwidth. Because DASH concurrently delivers different experts over the Direct and Relay routes, it can better utilize the additional HBF-side data availability enabled by larger transfers or lower read latency, translating device-level improvements into further end-to-end gains.

The HBF write-latency sweep reports total program-busy time, which measures the time spent programming HBF, and E2E-exposed stall, the portion not overlapped with computation or data movement. DASH hides all program time through 500 μs by scheduling writes alongside independent work, so the exposed stall remains zero even as program-busy time increases. At 5 ms, programming exceeds the available overlap window, adding 19.94 s of stall over the full execution. Thus, raw HBF program latency does not directly translate into serving delay; only the unhidden portion affects E2E latency.

For per-route D2D bandwidth, results are normalized to the 1.6 TB/s configuration, using the published HBF stack-level read-bandwidth target as the D2D-link provisioning point [37]. NAND-read timing and SRAM readiness remain separately modeled. E2E latency increases as D2D bandwidth

decreases below 1.6 TB/s, indicating that the link can become a bottleneck after data become ready in SRAM. Conversely, increasing bandwidth beyond 1.6 TB/s further reduces E2E latency, demonstrating that DASH can translate additional GPU-facing D2D bandwidth into end-to-end performance gains.

G. Cost Sensitivity and Interconnect Scalability

Because the public HBF fact sheet reports neither a numerical per-stack price nor package-integration cost [37], we present a parametric nominal-capacity-per-cost analysis of the memory subsystem rather than an absolute cost estimate. Let C_{HBM} denote the cost of one 24-GB HBM3E stack, let $r = C_{\mathrm{HBF}}/C_{\mathrm{HBM}}$ denote the HBF-to-HBM per-stack cost ratio, and let $\delta = \Delta C_{\mathrm{int}}/C_{\mathrm{HBM}} \geq 0$ denote DASH-specific incremental integration cost relative to the four-HBM reference, including additional peer PHYs and base-die logic, package integration, and yield loss not already included in the stack costs. Four 24-GB HBM3E stacks provide 96 GB of nominal capacity, whereas two 24-GB HBM3E stacks and two 512-GB HBF stacks provide 1,072 GB [25], [37]. The resulting reference-normalized nominal-capacity-per-cost gain is $G(r,\delta) = 44.67/(2 + 2r + \delta)$. We use $r = 1$ only as an illustrative anchor for the fact sheet's qualitative similar-cost statement, not as a vendor-quoted price point [37]. The fact sheet reports an HBM4-like footprint and stack height, motivating our equal-stack-count comparison [37]. At the illustrative $(r,\delta) = (1,0)$ anchor, $G = 11.17\times$, while $G > 1$ whenever $2r + \delta < 42.67$, equivalently $r < 21.33$ for $\delta = 0$.

For n HBM–HBF pairs, DASH uses $2n$ GPU-facing stack connections—the same number as a $2n$-HBM reference—and adds n pair-local HBM–HBF links, yielding $3n$ logical links and $O(n)$ link growth. Each additional pair adds HBF capacity together with a one-hop direct path and a two-hop pair-local relay path, without centralizing relay traffic or rewiring existing pairs. In the evaluated four-stack floorplan, facing-edge UCIe-A PHY placement keeps every UCIe mainband channel within the target advanced-package reach limit [36], while larger configurations remain subject to GPU-edge PHY beachfront, package area, and interposer routability.

H. HBF Endurance

We project HBF endurance from event-level logical KV writes generated by the evaluated serving workload, rather than from accelerated-aging measurements. Following prior work [21], we assume $E_{\mathrm{SLC}} = 100{,}000$ program/erase cycles. The projected lifetime is

$$\mathrm{Lifetime} = \frac{C_{\mathrm{KV}} E_{\mathrm{SLC}}}{A_W R_{\mathrm{HBF}} u}, \tag{6}$$

where C_{KV} is writable KV capacity, R_{HBF} is the logical HBF write rate, A_W is the physical-to-logical write-amplification factor (WAF) [12], and u is the duty factor. Using the request-stateful continuous-batching scheduler described in Section VI-E, we evaluate a prefill-heavy, write-stress case with Llama 4 Maverick at $L_{\mathrm{in}}/L_{\mathrm{out}}$=32K/128 and $B_{\max}$=32, where long prefills dominate HBF KV writes. After warm-up, 256 completed requests generate 1,648.34 GB of logical HBF writes over 1,623.76 s, yielding R_{HBF}=1,015.13 MB/s; every measured iteration contains both prefill and decode work, and the 128- and 256-request estimates differ by only 0.0064%. The run remains within the 1,024-GB HBF capacity through capacity-aware admission. After 801.42 GB of fixed allocation and a 16-GB reserve for garbage collection, C_{KV}=206.58 GB. This gives a 0.645-year continuous-activity projection at A_W=1 and u=1, assuming uniform wear and no bad-block loss. All measured HBF writes are page-aligned prefill writes: decode KV remains in HBM, and programmed bytes match logical bytes, giving a modeled page-padding factor of 1.0. Thus, sustained long-prompt endurance is governed by the rate at which prefill KV cycles through the writable region, rather than nominal HBF capacity; DASH confines this unavoidable traffic to large aligned transfers without adding token-granular decode programs or page-padding overhead. A deployed-lifetime claim additionally requires target-device measurements of total WAF, per-block erase counts, bad-block growth, and device aging, together with a deployment-specific duty cycle [24], [39].

VII. Related Work

Prior work has largely treated HBF organization and serving scheduling as separate concerns. H^3 places large read-only data in HBF and mutable state in HBM [9]; DASH goes beyond this placement split by serving HBF-resident data over concurrent Direct and Relay paths. A recent study analyzes the opportunities and challenges of using HBF as GPU memory for LLM inference [45], while complementary work evaluates HBF-resident KV cache performance and endurance [21]; DASH additionally coordinates latency-critical expert delivery with prefill- and decode-specific KV movement. An HBM–HBF pooling architecture places HBF behind HBM and uses a custom base die for layer-wise prefetching and buffered writeback [33]; DASH instead retains direct GPU–HBF access and uses the HBM relay path concurrently. Within this topology, DASH distinguishes four execution modes, matching each mode to its bandwidth and access granularity. Recent expert-prefetching methods predict or speculate future expert selections but do not guarantee the exact top-k decision [22], [55]; DASH provides exact early selection by reformulation of bias-free router. Unlike Sarathi-Serve, which coordinates prefill and decode without modeling HBF transfers or program operations [1], DASH jointly schedules continuous batching, HBF paths, and writes for dynamic MoE serving.

VIII. Conclusion

This paper presents DASH, a GPU memory architecture that connects HBM and HBF as main memory tiers for LLM inference. A dedicated HBM–HBF path enables HBF-resident data to reach the GPU either directly or through HBM. DASH combines this dual-path access with workload-aware data placement, early expert determination, and prefill/decode-aware KV write scheduling to hide HBF access latency while avoiding interference with latency-critical reads. DASH also addresses the endurance challenges of flash-based memory by

managing KV cache writes as part of the serving schedule. By using HBF as a main memory rather than relying solely on costly HBM capacity, DASH provides a more cost-effective and scalable system to support larger models and longer KV caches.

REFERENCES

[1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 117–134.

[2] N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, and R. Panigrahy, "Design tradeoffs for SSD performance," in *2008 USENIX Annual Technical Conference (USENIX ATC 08)*. USENIX Association, 2008, pp. 57–70. [Online]. Available: https://www.usenix.org/conference/2008-usenix-annual-technical-conference/design-tradeoffs-ssd-performance

[3] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 4895–4901. [Online]. Available: https://aclanthology.org/2023.emnlp-main.298/

[4] D. Das Sharma, S. Choudhary, P. Onufryk, and R. Pelt, "On-Package Memory with Universal Chiplet Interconnect Express (UCIe): A Low Power, High Bandwidth, Low Latency and Low Cost Approach," in *Proceedings of the IEEE Symposium on High-Performance Interconnects (HOTI)*, 2025, pp. 87–96. [Online]. Available: https://arxiv.org/abs/2510.06513

[5] DeepSeek-AI, "DeepSeek-V4-Pro model card," Hugging Face, 2026, accessed: 2026-07-28. [Online]. Available: https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro

[6] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. L. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, X. Wang, X. Liu, X. Xie, X. Yu, X. Song, X. Zhou, X. Yang, X. Lu, X. Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Zheng, Y. Zhang, Y. Xiong, Y. Zhao, Y. He, Y. Tang, Y. Piao, Y. Dong, Y. Tan, Y. Liu, Y. Wang, Y. Guo, Y. Zhu, Y. Wang, Y. Zou, Y. Zha, Y. Ma, Y. Yan, Y. You, Y. Liu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Huang, Z. Zhang, Z. Xie, Z. Hao, Z. Shao, Z. Wen, Z. Xu, Z. Zhang, Z. Li, Z. Wang, Z. Gu, Z. Li, and Z. Xie, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024. [Online]. Available: https://arxiv.org/abs/2405.04434

[7] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024. [Online]. Available: https://arxiv.org/abs/2412.19437

[8] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022. [Online]. Available: https://jmlr.org/papers/v23/21-0998.html

[9] M. Ha, E. Kim, and H. Kim, "H 3: Hybrid architecture using high bandwidth memory and high bandwidth flash for cost-efficient llm inference," *IEEE Computer Architecture Letters*, 2026.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[11] D. Hong, M. Kim, G. Cho, D. Lee, and J. Kim, "GuardedErase: Extending SSD lifetimes by protecting weak wordlines," in *20th USENIX Conference on File and Storage Technologies (FAST 22)*. USENIX Association, 2022, pp. 133–146. [Online]. Available: https://www.usenix.org/conference/fast22/presentation/hong

[12] X.-Y. Hu, E. Eleftheriou, R. Haas, I. Iliadis, and R. Pletka, "Write amplification analysis in flash-based solid state drives," in *Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference*. ACM, 2009, pp. 10:1–10:9.

[13] H. Huang, N. Ardalani, A. Sun, L. Ke, H.-H. S. Lee, S. Bhosale, C.-J. Wu, and B. Lee, "Toward efficient inference for mixture of experts," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 84 033–84 059. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/98bf3b8505c611ac21055dd9d355c66e-Abstract-Conference.html

[14] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, J. Chau, P. Cheng, F. Yang, M. Yang, and Y. Xiong, "Tutel: Adaptive mixture-of-experts at scale," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 5, 2023. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2023/hash/5616d34cf8ff73942cfd5aa922842556-Abstract-mlsys2023.html

[15] Intel Corporation, "Intel xeon platinum 8452y processor specifications," https://www.intel.com/content/www/us/en/products/sku/231761/intel-xeon-platinum-8452y-processor-67-5m-cache-2-00-ghz/specifications.html, 2023, accessed July 28, 2026.

[16] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural computation*, vol. 3, no. 1, pp. 79–87, 1991.

[17] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. Bou Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. Renard Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. Le Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024. [Online]. Available: https://arxiv.org/abs/2401.04088

[18] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: Cpu–gpu orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=N5fVv6PZGz

[19] Kimi Team, "Kimi K2: Open agentic intelligence," *arXiv preprint arXiv:2507.20534*, 2025. [Online]. Available: https://arxiv.org/abs/2507.20534

[20] KIOXIA Corporation, *TC58CYG2S0HRAIJ: 4Gb 1.8V Serial Interface NAND Technical Data Sheet*, KIOXIA Corporation, Oct. 2019, revision 2.00; programming, reading, and erasing characteristics in Section 3.7; accessed 2026-07-24. [Online]. Available: https://europe.kioxia.com/content/dam/kioxia/newidr/productinfo/datasheet/201910/DST_TC58CYG2S0HRAIJ-TDE_EN_36007.pdf

[21] K. Kyung, Y. J. Moon, J. Cho, and J. H. Ahn, "High-bandwidth flash for KV caches: Endurance and performance implications," *IEEE Computer Architecture Letters*, vol. 25, no. 1, pp. 210–213, Jan. 2026.

[22] V. Madan, P. Singhania, A. Bhatele, T. Goldstein, and A. Panda, "Speculating experts accelerates inference for mixture-of-experts," *arXiv preprint arXiv:2603.19289*, 2026. [Online]. Available: https://arxiv.org/abs/2603.19289

[23] Meta AI, "Llama 4 Maverick 17B-128E model card and configuration," 2025, accessed: 2026-07-14. [Online]. Available: https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E-Instruct

[24] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "A large-scale study of flash memory failures in the field," in *Proceedings of the 2015 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. ACM, 2015, pp. 177–190. [Online]. Available: https://doi.org/10.1145/2745844.2745848

[25] Micron Technology, Inc., "Micron HBM3E Product Brief," Micron Technology, Inc., Product Brief Rev. C, Oct. 2023, accessed: Jul. 31, 2026. [Online]. Available: https://assets.micron.com/adobe/assets/urn%3Aaaid%3Aaem%3Ab710d8f2-7f66-44c1-a234-456e2b986347/renditions/original/as/hbm3e-product-brief.pdf

[26] Mistral AI, "Mixtral-8x22B-v0.1 model card and configuration," 2024, accessed: 2026-07-14. [Online]. Available: https://huggingface.co/mistralai/Mixtral-8x22B-v0.1

[27] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021. [Online]. Available: https://cs.stanford.edu/people/matei/papers/2021/sc_megatron_lm.pdf

[28] NVIDIA, *NVIDIA H100 PCIe GPU Product Brief*, NVIDIA Corporation, 2022. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/gtcs22/data-center/h100/PB-11133-001_v01.pdf

[29] NVIDIA, "Nvidia h100 tensor core gpu," https://www.nvidia.com/en-us/data-center/h100/, 2024, accessed: 2026-07-21.

[30] NVIDIA Corporation, "NVIDIA Grace Hopper Superchip Architecture In-Depth," Nov. 2022, accessed: 2026-07-24. [Online]. Available: https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/

[31] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture," Architecture whitepaper, version 1.04, 2023, tables 1 and 3 report 756 dense and 1,513 sparse BF16 Tensor TFLOP/s with FP32 accumulation for H100 PCIe; accessed 2026-07-19. [Online]. Available: https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c

[32] NVIDIA Corporation, "NVIDIA HGX B200 reduces embodied carbon emissions intensity," NVIDIA Technical Blog, Sep. 2025, reports 180 GB of HBM3e memory per NVIDIA B200 GPU; accessed 2026-07-24. [Online]. Available: https://developer.nvidia.com/blog/nvidia-hgx-b200-reduces-embodied-carbon-emissions-intensity/

[33] J. Park, H. An, H. Suh, Y. Yoon, H. Lee, and J. Kim, "Hbm-hbf-centric memory pooling architecture with custom base die for terabyte-scale llm inference," *IEEE Computer Architecture Letters*, 2026.

[34] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems*, vol. 5, 2023. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html

[35] Qwen Team, "Qwen3-235B-A22B: Model announcement and configuration," 2025, accessed: 2026-07-14. [Online]. Available: https://qwenlm.github.io/blog/qwen3/

[36] S. Rusu, "How UCIe will enable broad chiplet adoption," UCIe Consortium, Mar. 2023, describes silicon-interposer and bridge implementations of UCIe advanced packaging with channels up to 2 mm; accessed Jul. 31, 2026. [Online]. Available: https://www.uciexpress.org/post/how-ucie-will-enable-broad-chiplet-adoption

[37] SanDisk Corporation, "High Bandwidth Flash (HBF) Fact Sheet," SanDisk Corporation, Tech Brief, Jul. 2025, tech brief; first-generation target of 512 GB capacity and 1.6 TB/s read bandwidth per 16-die stack; accessed Jul. 31, 2026. [Online]. Available: https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/collateral/company/Sandisk-HBF-Fact-Sheet.pdf

[38] SCALE Lab, Seoul National University, "LLMSimulator," GitHub repository, Oct. 2025, git commit 419252761fbdb95b789778a02256d458a5537ec7; accessed July 31, 2026. [Online]. Available: https://github.com/scale-snu/LLMSimulator

[39] B. Schroeder, R. Lagisetty, and A. Merchant, "Flash reliability in production: The expected and the unexpected," in *14th USENIX Conference on File and Storage Technologies (FAST 16)*. USENIX Association, 2016, pp. 67–80.

[40] D. D. Sharma and T. M. Coughlin, "Universal chiplet interconnect express: An open industry standard for memory and storage applications," *Computer*, vol. 57, no. 1, pp. 75–81, Jan. 2024.

[41] D. D. Sharma, G. Pasdast, Z. Qian, and K. Aygun, "Universal chiplet interconnect express (ucie): An open industry standard for innovations with chiplets at package level," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 12, no. 9, pp. 1423–1431, 2022.

[42] D. D. Sharma, G. Pasdast, S. Tiagaraj, and K. Aygün, "High-performance, power-efficient three-dimensional system-in-package designs with universal chiplet interconnect express," *Nature Electronics*, vol. 7, pp. 244–254, 2024.

[43] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019. [Online]. Available: https://arxiv.org/abs/1911.02150

[44] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=B1ckMDqlg

[45] D. Son, Y. Park, H. Cho, H. Ham, O. Mutlu, S. Lee, G. Kim, and J. Park, "Exploring High-Bandwidth Flash for Modern LLM Inference: Opportunities and Challenges," *IEEE Computer Architecture Letters*, vol. 25, no. 2, pp. 251–254, 2026.

[46] UCIe Consortium, *Universal Chiplet Interconnect Express (UCIe) Specification, Revision 3.0*, UCIe Consortium, Aug. 2025, supports 48- and 64-GT/s data rates; accessed July 29, 2026. [Online]. Available: https://www.uciexpress.org/specifications

[47] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need

[48] Z. Wu and J.-R. Guo, "Analysis of UCIe 48/64-GT/s electrical links," *IEEE Open Journal of the Solid-State Circuits Society*, vol. 5, pp. 401–409, 2025.

[49] xAI, "Grok-1 open-weight model repository," 2024, accessed: 2026-07-14. [Online]. Available: https://github.com/xai-org/grok-1

[50] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Offloading-efficient MoE model serving," *arXiv preprint arXiv:2401.14361*, 2024. [Online]. Available: https://arxiv.org/abs/2401.14361

[51] X. Yang, D. Dutta, Y. Li, and M. Higashitani, "High bandwidth nonvolatile memory devices," U.S. Patent Application Publication US 2025/0259685 A1, assigned to SanDisk Technologies LLC, 2025, accessed: 2026-07-18. [Online]. Available: https://patents.google.com/patent/US20250259685A1/en

[52] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[53] Z.ai, "GLM-5 model card," Hugging Face, 2026, accessed: 2026-07-28. [Online]. Available: https://huggingface.co/zai-org/GLM-5

[54] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 12 360–12 371. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html

[55] S. Zhu, S. Bohl, R. Oester, and G. Alonso, "Pre-attention expert prediction and prefetching for mixture-of-experts large language models," *arXiv preprint arXiv:2511.10676*, 2025. [Online]. Available: https://arxiv.org/abs/2511.10676